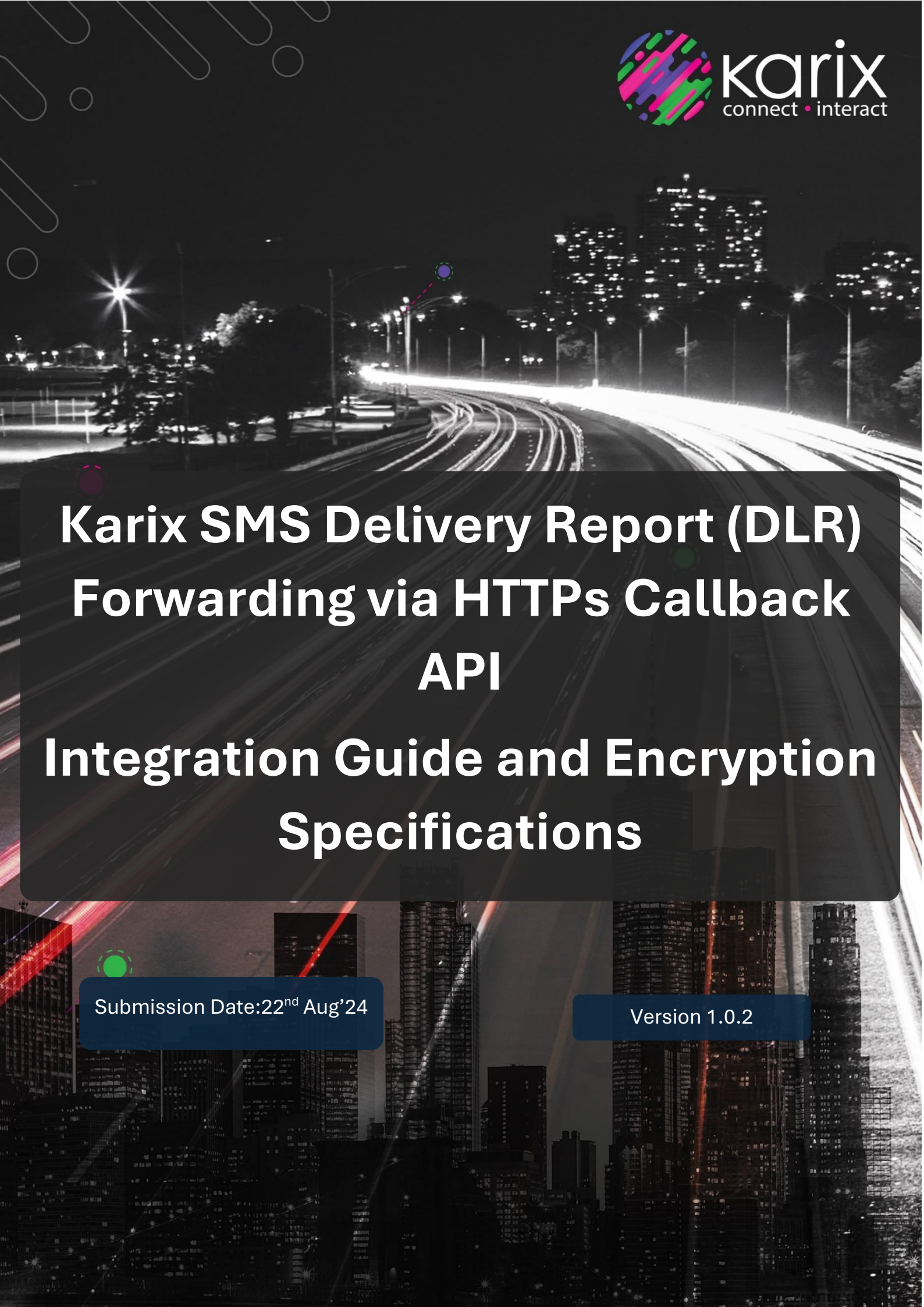




Karix SMS Delivery Report (DLR) Forwarding via HTTPs Callback API Integration Guide and Encryption Specifications



Submission Date: 22nd Aug'24

Version 1.0.2

Contents

1. Introduction	3
2. SMS Delivery Report (DLR) Forwarding Formats	3
2.1 Plain HTTPs Callback API (JSON Format).....	3
2.2 Encrypted HTTPs Callback API (AES GCM Mode)	3
3. API Integration Details	3
3.1 Plain HTTPs Callback API (JSON Format).....	3
3.1.1 API Endpoint	3
3.1.2 API Headers	3
3.1.3 API Body Structure	3
3.1.4 Parameter Descriptions	4
3.2 Encrypted HTTPs Callback API (AES GCM Mode)	5
3.2.1 Encryption Specifications.....	5
3.2.2 API Body Structure	5
3.2.3 AES GCM Encryption Code Snippet.....	5
4. Justification for AES GCM Encryption	6
5. Conclusion	6

1. Introduction

This document provides a comprehensive overview of the SMS Delivery Report (DLR) forwarding process in real-time to the customer's HTTPs Callback (Webhook) API. The document outlines both plain and encrypted formats for delivering DLRs, based on the requirements specified by customers.

The recommended approach is to forward the DLRs using an HTTPs Callback API in JSON format, which is widely accepted for its simplicity and compatibility.

2. SMS Delivery Report (DLR) Forwarding Formats

2.1 Plain HTTPs Callback API (JSON Format)

We support forwarding SMS Delivery Reports via a plain HTTPs Callback API using the JSON format, allowing for seamless integration with customer systems.

2.2 Encrypted HTTPs Callback API (AES GCM Mode)

For enhanced security, we also offer the option to encrypt the entire JSON payload body containing the SMS delivery status using AES GCM mode encryption.

3. API Integration Details

3.1 Plain HTTPs Callback API (JSON Format)

3.1.1 API Endpoint

`https://<domain>/path`

3.1.2 API Headers

Content-Type: application/json

Authorization: Basic <space><Static Token>

3.1.3 API Body Structure

```
{
  "acode": "$acodelabel$",
  "pcode": "$pcodelabel$",
  "ackid": "$fileidlabel$",
  "mid": "$midlabel$",
  "dest": "$destlabel$",
  "send": "$sendlabel$",
  "stime": "$stimelabel$",
  "dtime": "$dtimelabel$",
  "status": "$statuslabel$",
  "Statusflag": "$statusflaglabel$",
  "reason": "$reasonlabel$",
  "type": "$typelabel$",
  "msg": "$messagelabel$"
```

```

"Splitmsgno": "$splitmsgpartslabel$",
"Splitmsgpatrs": "$splitmsgpartnolabel$",
"operator": "$operatorlabel$",
"circle": "$circlelabel$",
"cust_mid": "$custmidlabel$",
"tag1": "$tag1$",
"tag2": "$tag2$",
"tag3": "$tag3$",
"tag4": "$tag4$",
"tag5": "$tag5$",
"tag6": "$tag6$",
"tag7": "$tag7$",
"tag8": "$tag8$",
"tag9": "$tag9$",
"tag10": "$tag10$",
"tag11": "$tag11$",
"tag12": "$tag12$",
"tag13": "$tag13$",
"tag14": "$tag14$",
"tag15": "$tag15$",
"tag16": "$tag16$",
"tag17": "$tag17$",
"tag18": "$tag18$",
"tag19": "$tag19$",
"tag20": "$tag20$"
}

```

3.1.4 Parameter Descriptions

- **acode:** The account username created for the customer by Karix.
- **pcode:** The parent account username.
- **ackid:** The unique ID generated by Karix in the Send SMS API response.
- **dest:** The mobile number.
- **send:** The Sender ID (Header Name).
- **stime:** The time the message was submitted to the telecom operator. EPOCH format is also supported.
- **dtime:** The time the message was delivered.
- **status:** The SMS delivery status code.
- **Statusflag:** The SMS delivery status flag (Success, Failed, Rejected).
- **reason:** The SMS delivery status reason (Delivered, Network Failed, DLT Failed).
- **msg:** The message content.
- **Splitmsgno:** The multipart message content part number.
- **Splitmsgpatrs:** The multipart message content part number.
- **operator:** The mobile number subscriber's operator.
- **circle:** The mobile number subscriber's circle.
- **cust_mid, tag1 to tag20:** Unique identifiers passed by the customer in the Send SMS API request.

3.2 Encrypted HTTPs Callback API (AES GCM Mode)

3.2.1 Encryption Specifications

- **aes_mode:** GCM
- **key:** XXXXXXXXXXXXXXXXXXXX
- **iv_key:** XXXXXXXXXXXXXXXXXXXX
- **padding_method:** AES/GCM/NoPadding
- **encrypt_payload_format:** JSON
- **api_param_name:** payload
- **encrypt_output_text_format:** BASE64
- **gcm_tag_length:** 16

3.2.2 API Body Structure

```
{
  "aes_mode": "GCM",
  "key": "XXXXXXXXXXXXXXXXXX",
  "iv_key": "XXXXXXXXXXXXXXXXXX",
  "padding_method": "AES/GCM/NoPadding",
  "encrypt_payload_format": "JSON",
  "api_param_name": "payload",
  "encrypt_output_text_format": "BASE64",
  "gcm_tag_length": "16"
}
```

3.2.3 AES GCM Encryption Code Snippet

The following code snippet demonstrates how to perform AES GCM encryption using the provided data, key, initialization vector (IV), padding scheme, and GCM tag length.

```
public static String encryptMasterForGCM(String data, String key, String initVector, String
padding,
    String gcmTagLength) {
    String encryptedValue = "";
    int GCM_TAG_LENGTH = Integer.parseInt(gcmTagLength);

    try {
        GCMPParameterSpec gcmParameterSpec = new
GCMPParameterSpec(GCM_TAG_LENGTH * 8, initVector.getBytes());
        SecretKeySpec skp = new SecretKeySpec(key.getBytes("UTF-8"), "AES");
        Cipher cipher = Cipher.getInstance(padding);
        cipher.init(Cipher.ENCRYPT_MODE, skp, gcmParameterSpec);
        byte[] encrypted = cipher.doFinal(data.getBytes());

        encryptedValue = Base64.getEncoder().encodeToString(encrypted);

    } catch (Exception e) {
        e.printStackTrace();
    }
    return encryptedValue;
}
```

This method encrypts the input data using AES in GCM mode. The initialization vector (IV) and GCM tag length are provided as parameters to ensure secure encryption. The final encrypted value is returned as a Base64-encoded string.

4. Justification for AES GCM Encryption

AES GCM (Galois/Counter Mode) encryption is a widely accepted standard for securing sensitive data due to its strong security features. It provides both data confidentiality and integrity by combining encryption with authentication. This mode of encryption has been approved by our multiple client's InfoSec team for its ability to ensure that the SMS Delivery Reports remain secure during transmission. By encrypting the entire JSON payload, we minimize the risk of data breaches and unauthorized access, providing an additional layer of security in line with industry best practices.

5. Conclusion

This document serves as a guide for integrating the SMS Delivery Report (DLR) forwarding service via Karix's HTTPs Callback API. We support both plain and encrypted formats to meet varying security and integration requirements. Please refer to the provided details to ensure successful implementation.

-----End of Document-----